

We Tried to Prove Intelligent Test Selection Beats a Path Rule. The Answer Was Complicated.

What change-aware test selection is worth when the baseline is a cheap heuristic rather than the full suite.

Every test-selection tool ships with the same chart. On the left, the full suite: hundreds of test files, ten minutes, a wall of green. On the right, the selected subset: three files, twenty seconds. The percentage in between is the product.

We built one of these tools. DiffCI builds a real TypeScript dependency graph for a repository, works out which test files are reachable from the files a commit changed, and proposes running only those. Over the last two weeks of August 2026 we ran a sequence of pre-registered experiments to find out whether that chart is telling the truth. We published everything we found under a CC BY 4.0 licence, including the parts that argue against us. This is what the data says.

The obvious benchmark is wrong

The chart compares the selection against running everything. That comparison is almost impossible to lose. Any selector that runs fewer tests than "all of them" looks good against it, including one that picks test files at random and skips the rest.

The problem is that nobody's real CI runs everything, either, if they have thought about it for five minutes. Most teams already have some rule: run the package that changed, run the directory that changed, skip the suite when only docs moved. Those rules are free. They need no dependency graph, no compiler, no analysis step, and they are already in the pipeline. A selector that beats "run everything" has cleared a bar that was lying on the floor.

So before we measured anything we wrote down a harder opponent.

Establish a cheap opponent

The opponent is a path rule: map each changed file's path to the test files that share its directory prefix, run those, fall back to the full suite on anything it cannot map. It costs nothing to compute. It is what a competent engineer would write in an afternoon and never think about again.

We froze it before any results existed and applied it unchanged to every repository in the study. In the CPU-accounting experiments we went further and charged it nothing for analysis, even though its selection is computed from the same file profile our graph build produces. DiffCI pays for every CPU-second of its own analysis. The comparator pays for none of it. If DiffCI wins under that allocation, the win is not an artefact of how shared cost was divided.

The question then becomes precise. Not "does DiffCI run fewer tests than all of them?" but "does the information DiffCI extracts from the dependency graph beat a free heuristic, after DiffCI has paid for extracting it?"

Run it across real open-source history

The largest experiment took 20 public TypeScript and JavaScript repositories, chosen for structural diversity before any were run, and replayed 100 real commits from each through a real orchestrator: 2,000 deltas in total, verified by direct database query, no duplicates. The methodology was frozen and hashed at a commit before the run started. One delta produced an impossible count during the run. We excluded it from the aggregates rather than patch the harness mid-benchmark on an unconfirmed root cause, and we say so in the report.

None of these repositories is a customer. None has installed DiffCI, been contacted about it, or endorsed anything. They are public projects with public CI, and their names are in the study because a result on an anonymous corpus is not a result anyone can check.

For each delta we classified, independently of what DiffCI did, whether a selective decision was even possible. Three buckets. Sixty percent of deltas touched a lockfile, a workflow file or a root configuration, and fell back to the full suite by rule on both sides. Ten percent were already handled optimally by the path rule; there was nothing left to select away. The remaining 29.8%, 565 deltas, were genuine opportunities where the two approaches could disagree.

The surprising result

Inside those 565 opportunities, DiffCI won. It selected fewer tests than the path rule on 94.3% of them, tied on the rest, and lost on none. The median reduction against the path rule inside that bucket was 95%. If you only look at the cases where the graph gets a chance, the chart at the top of this article is honest.

Across the whole corpus, weighted by test count, the aggregate reduction against the path rule was 4.2%.

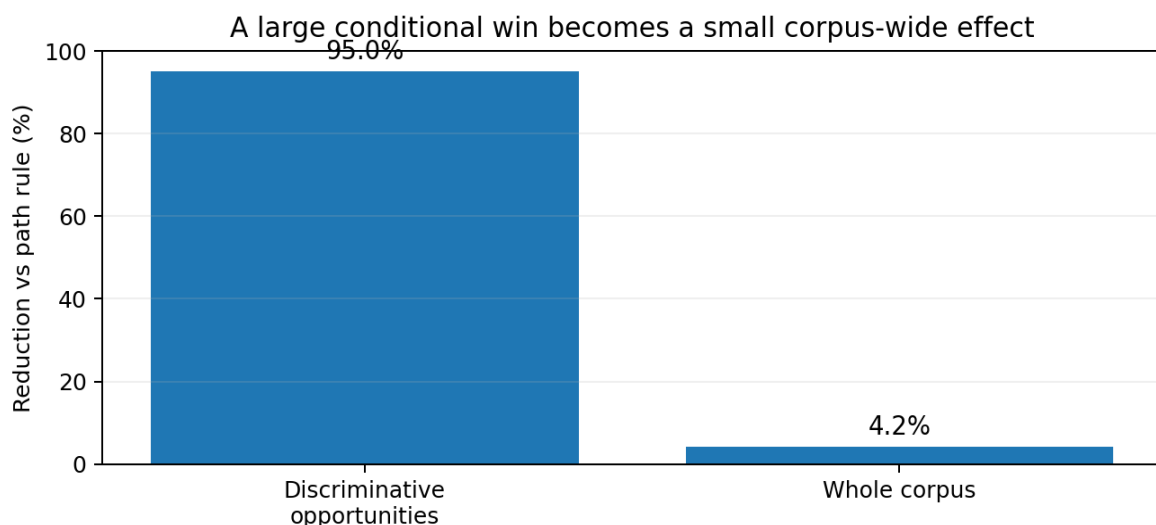


Figure 1. Conditional performance can look dramatic while the unconditional corpus-wide effect remains small.

Not 42. Four point two. The unconditional median task reduction was 0.0%. A repository-clustered bootstrap puts the 95% interval on that aggregate at -4.8% to $+47.9\%$, which crosses zero. On this sample we cannot rule out that the population effect is nothing.

Both numbers are true, and the gap between them is the finding. The advantage is large when it exists and it exists rarely, because most commits touch something the graph is not allowed to reason about. Seven of the twenty repositories fell back to the full suite on every single delta and contributed nothing but drag. The strongest correlate we found was repository size: the ten repositories below the median source-file count showed a 44.9% aggregate reduction, the nine above it showed -0.95% . Size and fallback rate are confounded in a sample of twenty, and we do not claim to have separated them.

An earlier 500-delta batch had put the aggregate at 24.6%. Scaling the corpus four times did not strengthen the result. It sharpened it into something narrower and more honest.

Safety complicates optimisation

Fewer tests only counts if the ones you skipped would have passed. Using authenticated GitHub Actions history for the same 2,000 deltas, we found 280 real job-level CI failures. DiffCI's plan would have missed ten of them. The path rule would have missed 82.

That is a job-level failure recall of 96.4% against 70.7%, and it is a large, real safety advantage. It is also ten misses. Earlier partial snapshots during the run had suggested zero, and it would have been easy to leave the story there. The final report refused to round to it. A selector that is right 96.4% of the time is a selector that is wrong on one commit in twenty-eight that would have broken the build, and any team adopting one needs to know that number before they know the savings.

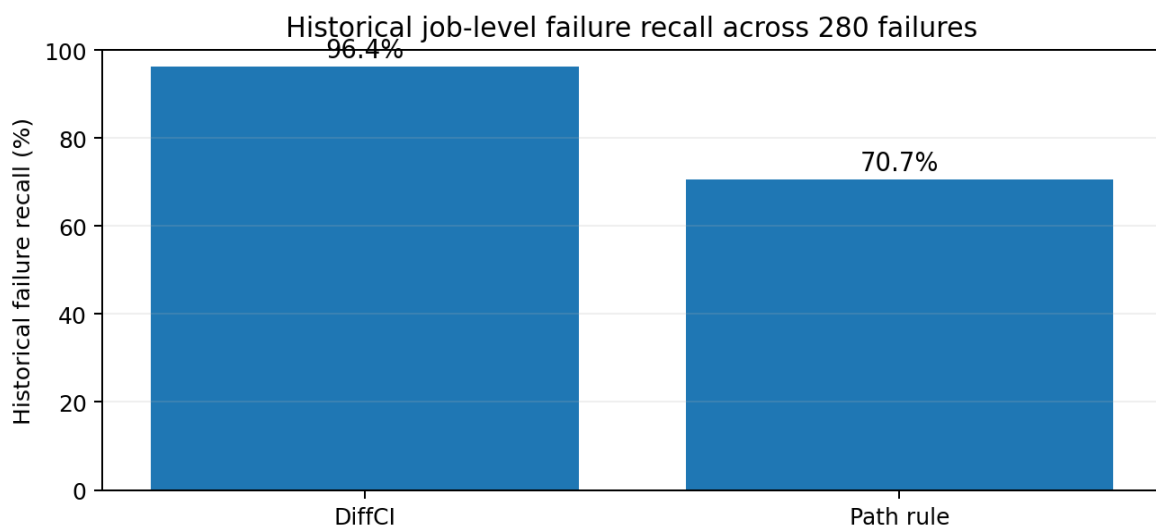


Figure 2. Retrospective historical failure recall. DiffCI still missed 10 of 280 failures; the path rule missed 82.

Where we could manufacture a regression and measure whether it was caught, the record is cleaner: across five corpora, every one of the 32 measurable mutations was detected, with no false greens. We report that as a count, not a rate. Thirty-two cases do not support a reliability estimate, and the frozen bundle that holds twenty-five of them prints that caveat itself.

Test reduction is not CI reduction

A computed selection is a claim. On two repositories we tested it by actually executing both paths at the same commit, the full suite and DiffCI's subset, and timing them: ten merges in all, with a runtime check that the test runner honoured the selection instead of quietly broadening it. It did on twelve of thirteen executions. The thirteenth was withheld rather than counted.

On cal.com, five merges chosen by a rule written down before their costs were known produced a test-stage reduction of 86% to 91.6%. Measured against the whole CI job, the same selection came to 44.2%. The reason is boring and decisive: cal.com spends 319.7 seconds on `yarn install` and 17.1 more on `prisma generate` before a single test runs, and those seconds are paid identically whether you run 250 test files or one.

On deepseek-harness, whose install takes 23 to 39 seconds, the same selection quality produced 79.5% to 89.5% at job level.

That forty-point gap between two repositories has nothing to do with how good the selection was. It is entirely the ratio of install to test. Anyone quoting a single industry percentage for what test selection saves has not measured the repository they are quoting it to. And a reader can find their own ceiling in about a minute: open a recent CI run, note how long install and setup took, and note how long tests took. If install dominates, selection cannot help much no matter how smart it is.

Intelligence has a cost

Here is the experiment that made us trust the meter. On honojs/hono, measured in a canonical Linux container across 22 candidates, DiffCI's gross saving against the full suite was 1,408.92 CPU-seconds. Its result against the path rule, after paying for its own analysis, was -80.90.

DiffCI lost. Four candidates positive, eighteen negative. Every one of the four wins was a case where the path rule blew up to 123 test files and DiffCI held at 83. Everywhere the path rule was already tight, one to nineteen files, DiffCI selected about as many and paid a fixed toll of roughly 3.3 CPU-seconds of analysis on top. On one commit both sides selected exactly one test, both cost 1.90 CPU-seconds to run, and DiffCI still lost by 3.06. Entirely to overhead.

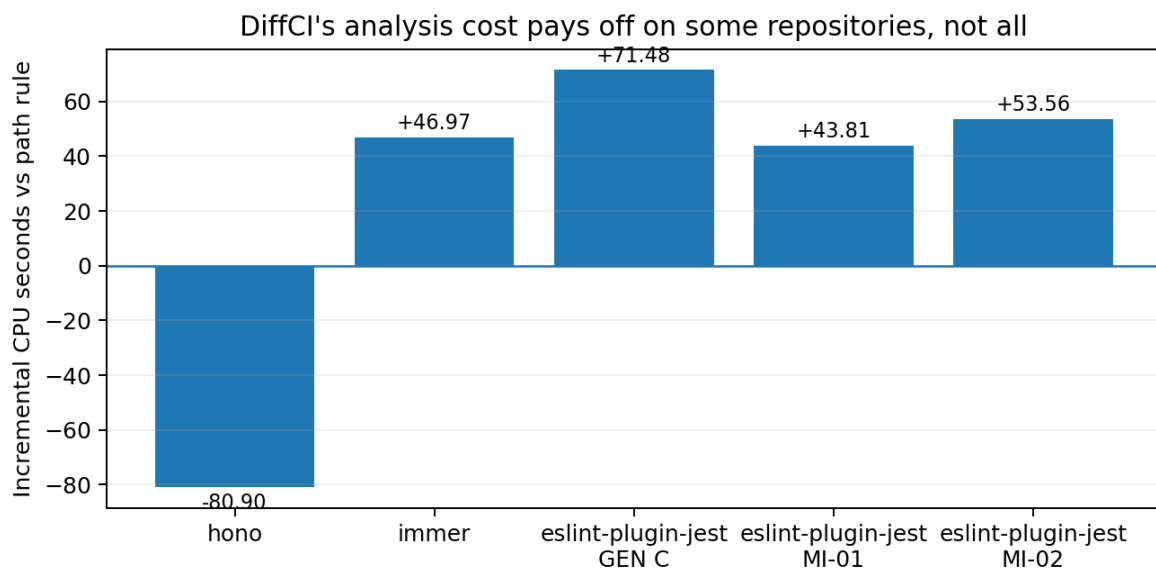


Figure 3. Positive means DiffCI's intelligence paid for itself against the free path-rule comparator after analysis cost.

The test counts had predicted this: twelve of twenty hono selections were classed as over-broad before any CPU was measured. Then an independent instrument, the CPU meter, said the same thing. A meter that reported DiffCI winning everywhere would have indicted itself before the result could be interesting.

The honest characterisation is not that DiffCI is worse than a path rule. It is that on hono, DiffCI pays a small fixed premium on every commit to insure against the path rule occasionally selecting 123 tests, and on hono the insurance costs more than it pays out. On zod the sign flipped: across five confirmed cases DiffCI selected 629 tests where the path rule selected 761 and the full suite held 967. On eslint-plugin-jest three blind-drawn candidates cost DiffCI 4 to 6 CPU-seconds where the path rule cost 52 to 77. Whether the premium is worth it is a property of the repository, not of the tool.

Graph sophistication is not automatically valuable

There is one more opponent, cheaper than the path rule, and it beat us to a draw.

On the single blind-drawn candidate where detection, graph contribution and economics were all measurable at once, DiffCI selected two files out of 174 against the path rule's 164, caught the injected regression, and saved 71.48 CPU-seconds. Both selected files were files the commit had changed. Nothing was reached through the dependency graph. A rule that says "run only the files this commit touched" would have produced the identical selection, the identical detection and the identical cost, with no graph at all.

On two other candidates the graph did reach a test file the commit had not touched, selecting two of 159 against the path rule's 149. On both, detection was unmeasurable: reverting the change did not fail the full suite, so there was nothing to catch. The evidence ladder therefore closes with one measured safety-plus-savings result, two independent demonstrations that the graph reaches things the diff cannot name, and no case where both were measured together.

We have not shown that the graph is causally necessary for anything. We have written that sentence into our own standard: beating the full suite no longer counts, beating the path rule is necessary but not sufficient, and any claim that the graph earns its compute has to beat

direct-only, the heuristic that has tied twice.

The larger conclusion

Most of the repositories we looked at, we could not reason about at all. Of npm's forty most depended-upon packages, eight have no `tsconfig.json` and are refused before a graph is built. Of the 28 we could measure, twenty have fewer than one test file in five connected to any production file through the static import graph, and at least two of the four reasons for that are our own limitations, not the repositories'. Of five external repositories a third party picked for us before we had looked at them, one passed the pre-registered gates. On that one, immer, a rule frozen before the measurement predicted a positive sign and the measurement was positive: 46.97 CPU-seconds. That is one sign match. It validates the ordering of the experiment, not the predictor.

So here is where two weeks of trying to prove that intelligent test selection beats a path rule ended up.

The question is not how many tests a selector can avoid. Against "run everything", the answer is "most of them", and it is meaningless. The question is whether the information used to avoid them improves safety-adjusted CI economics by enough to justify the cost and the complexity of obtaining it. On the evidence we have, that answer is: sometimes, narrowly, on small single-package repositories with connected test suites and cheap installs, and provably not on others. The graph buys a real safety margin over a path rule, ten misses instead of 82, and it charges a real premium for it, which on some repositories exceeds what it saves.

That is a less satisfying chart. It is the only one we can defend.

Nothing in this article has ever run against anyone's production CI. Every saving here was measured by executing both paths in our own sandbox. As of 3 September 2026 there were zero external installs, confirmed by database query, and the study says so in a section titled "What this study does not establish" that we consider the most important page in it.

The DiffCI Open Evidence Study 2026 is published under CC BY 4.0 at diffci.com/research/diffci-open-evidence-2026, with every figure in this article in a CSV alongside its evidence level and the report that produced it.